

Explicação Geral do Código fonte do Projeto

O projeto utiliza Python com estruturas de dados e aprendizado de máquina para realizar a estratificação de pacientes e a análise de remédios.

A Árvore AVL foi implementada para armazenar e organizar os dados dos remédios de forma balanceada, servindo como base para a rotina de aprendizado de máquina. Ela garante eficiência nas operações de inserção e busca, com complexidade $O(\log n)$.

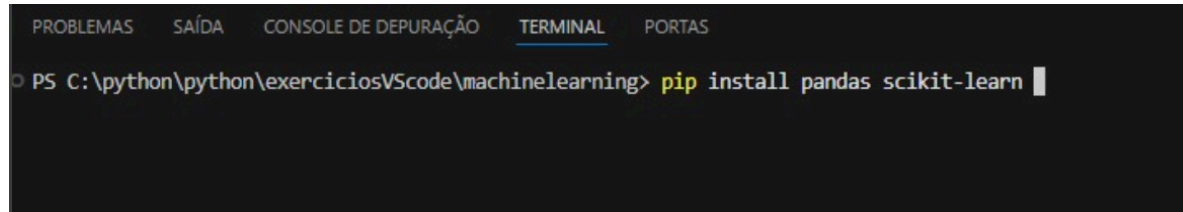
O Grafo é responsável por estruturar as informações dos pacientes e alimentar o modelo de aprendizado, que identifica e direciona automaticamente os pacientes prioritários com base na gravidade da doença. Ele também possui funções de fila para ordenar o atendimento dos mais graves.

O Main integra tudo: coleta os dados, executa o aprendizado de máquina (usando árvores de decisão do scikit-learn), realiza a detecção de outliers nos remédios e cria a fila de prioridade dos pacientes.

O código conta ainda com análise assintótica nas funções da AVL e do Grafo, e testes unitários, garantindo desempenho, precisão e confiabilidade.

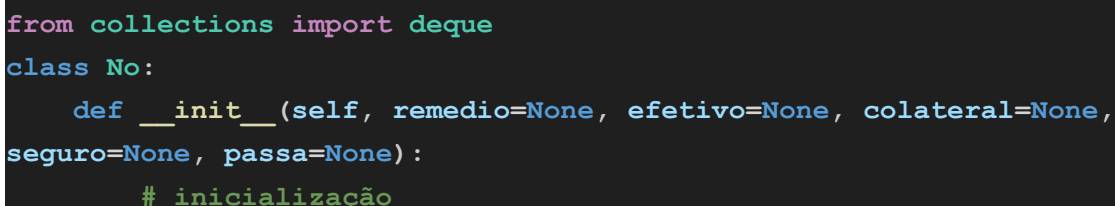
O projeto foi separado em 4 arquivos dentro de uma única pasta, para poder reutilizar código, se necessário, e para maior organização de tarefas.

Para poder rodar o código que vai ser mostrado logo em seguida precisa das instalações das bibliotecas: pandas e scikit-learn, como mostrado abaixo.



A screenshot of a terminal window with a dark background. The title bar at the top shows tabs for 'PROBLEMAS', 'SAÍDA', 'CONSOLE DE DEPURACÃO', 'TERMINAL' (which is active), and 'PORTAS'. The terminal text shows the command prompt 'PS C:\python\python\exerciciosVScode\machinelearning>' followed by the command 'pip install pandas scikit-learn' with a cursor at the end.

Código do arquivo: [avltree.py](#)



A screenshot of Python code from a file named 'avltree.py'. The code is as follows:

```
from collections import deque
class No:
    def __init__(self, remedio=None, efetivo=None, colateral=None,
seguro=None, passa=None):
    # inicialização
```

```

        self.key = (passa, remedio)
        self.remedio = remedio
        self.passa = passa
        self.efetivo = efetivo
        self.colateral = colateral
        self.seguro = seguro
        self.height = 0
        self.parent = None
        self.left = None
        self.right = None

class AVLarvore:
    def __init__(self):
        self.root = None

    # Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
    recursão
    def insert(self, remedio, efetivo, colateral, seguro, passa):
        if self.root is None:
            self.root = No(remedio, efetivo, colateral, seguro, passa)
        else:
            self._insert(remedio, efetivo, colateral, seguro, passa,
self.root)

    # Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
    recursão
    def _insert(self, remedio, efetivo, colateral, seguro, passa,
no_atual):
        new_key = (passa, remedio)
        current_key = (no_atual.passa, no_atual.remedio)

        if new_key < current_key:
            if no_atual.left is None:
                no_atual.left = No(remedio, efetivo, colateral,
seguro, passa)
                no_atual.left.parent = no_atual
                # Atualização altura do nó atual após inserção
                no_atual.height = 1 + max(
                    self.get_height(no_atual.left),
                    self.get_height(no_atual.right)
                )
                self._inspect_insertion(no_atual.left)

```

```

        else:
            self._insert(remedio, efetivo, colateral, seguro,
passa, no_atual.left)
        elif new_key > current_key:
            if no_atual.right is None:
                no_atual.right = No(remedio, efetivo, colateral,
seguro, passa)
                no_atual.right.parent = no_atual
                # Atualizar altura do nó atual após inserção
                no_atual.height = 1 + max(
                    self.get_height(no_atual.left),
                    self.get_height(no_atual.right)
                )
                self._inspect_insertion(no_atual.right)
            else:
                self._insert(remedio, efetivo, colateral, seguro,
passa, no_atual.right)
        else:
            print("Medicamentos já existem na arvore.")

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de recursão
def print_tree(self):
    if self.root is not None:
        self._print_tree(self.root)

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de recursão
def _print_tree(self, cur_node):
    if cur_node is not None:
        self._print_tree(cur_node.left)
        print(f'{cur_node.remedio}: passa={cur_node.passa},
efeito: {cur_node.efetivo}, colateral: {cur_node.colateral},
segurança: {cur_node.seguro} h={cur_node.height}')
        self._print_tree(cur_node.right)

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de recursão
def height(self):
    if self.root is not None:
        return self._height(self.root, 0)
    else:
        return -1

# Análise Assintótica Tempo: O(n) Espaço: O(h) - Pilha de recursão

```

```

def _height(self, no_atual, altura_atual):
    if no_atual is None:
        return altura_atual - 1 # Ajuste para consistência

    left_height = self._height(no_atual.left, altura_atual + 1)
    right_height = self._height(no_atual.right, altura_atual + 1)
    return max(left_height, right_height)

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def find(self, passa, remedio):
    if self.root is not None:
        return self._find((passa, remedio), self.root)
    else:
        return None

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def _find(self, key, atual):
    key_atual = (atual.passa, atual.remedio)

    if key == key_atual:
        return atual # retorna o nó inteiro
    elif key < key_atual and atual.left is not None:
        return self._find(key, atual.left)
    elif key > key_atual and atual.right is not None:
        return self._find(key, atual.right)
    return None

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def delete_value(self, passa, remedio):
    return self.delete_node(self.find(passa, remedio))

# Análise Assintótica Tempo: O(log n) Espaço: O(log n) - Pilha de
recursão
def delete_node(self, no):
    if no is None or self.find(no.passa, no.remedio) is None:
        print("No não achado na arvore.")
        return None

    def min_value_node(n):

```

```

        atual = n
        while atual.left is not None:
            atual = atual.left
        return atual

def num_children(n):
    nc = 0
    if n.left is not None:
        nc += 1
    if n.right is not None:
        nc += 1
    return nc

no_parent = no.parent
no_children = num_children(no)

if no_children == 0:
    if no_parent is not None:
        if no_parent.left == no:
            no_parent.left = None
        else:
            no_parent.right = None
    else:
        self.root = None

elif no_children == 1:
    child = no.left if no.left is not None else no.right

    if no_parent is not None:
        if no_parent.left == no:
            no_parent.left = child
        else:
            no_parent.right = child
    else:
        self.root = child

    child.parent = no_parent

else: # no_children == 2
    sucessor = min_value_node(no.right)

    # copiar dados do sucessor para o nó

```

```

        no.key = sucessor.key
        no.passa = sucessor.passa
        no.remedio = sucessor.remedio
        no.efetivo = sucessor.efetivo
        no.colateral = sucessor.colateral
        no.seguro = sucessor.seguro

        # remover o sucessor
        self.delete_node(sucessor)
        return

    if no_parent is not None:
        no_parent.height = 1 + max(
            self.get_height(no_parent.left),
            self.get_height(no_parent.right)
        )
        self._inspect_deletion(no_parent)

    # Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(\log n)$  - Pilha de
recursão
    def search(self, passa, remedio):
        if self.root is not None:
            return self._search((passa, remedio), self.root)
        else:
            return False

    # Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(\log n)$  - Pilha de
recursão
    def _search(self, key, no_atual):
        chave_atual = (no_atual.passa, no_atual.remedio)

        if key == chave_atual:
            return True
        elif key < chave_atual and no_atual.left is not None:
            return self._search(key, no_atual.left)
        elif key > chave_atual and no_atual.right is not None:
            return self._search(key, no_atual.right)
        return False

    # Análise Assintótica Tempo:  $O(\log n)$  Espaço:  $O(1)$ 
    def _inspect_insertion(self, no_atual):
        if no_atual.parent is None:

```

```

        return

    left_height = self.get_height(no_atual.left)
    right_height = self.get_height(no_atual.right)

    if abs(left_height - right_height) > 1:
        path = [no_atual.parent, no_atual]
        # garantir que há x (taller child of no_atual)
        y = path[1]
        x = self.taller_child(y)
        self._rebalance_node(path[0], y, x)
        return

    new_height = 1 + max(self.get_height(no_atual.left),
self.get_height(no_atual.right))
    if new_height != no_atual.parent.height:
        no_atual.parent.height = new_height

    self._inspect_insertion(no_atual.parent)

# Análise Assintótica Tempo: O(log n) Espaço: O(1)
def _inspect_deletion(self, no_atual):
    if no_atual is None:
        return

    left_height = self.get_height(no_atual.left)
    right_height = self.get_height(no_atual.right)

    if abs(left_height - right_height) > 1:
        y = self.taller_child(no_atual)
        x = self.taller_child(y)
        self._rebalance_node(no_atual, y, x)

    self._inspect_deletion(no_atual.parent)

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def _rebalance_node(self, z, y, x):
    if y == z.left and x == y.left:
        self._right_rotate(z)
    elif y == z.left and x == y.right:
        self._left_rotate(y)
        self._right_rotate(z)

```

```

elif y == z.right and x == y.right:
    self._left_rotate(z)
elif y == z.right and x == y.left:
    self._right_rotate(y)
    self._left_rotate(z)
else:
    raise Exception("Aconteceu um erro ao rebalancear.")

#Rotações da árvore para garantir uma árvore balanceada
# Análise Assintótica Tempo: O(1) Espaço: O(1)
def _right_rotate(self, z):
    sub_root = z.parent
    y = z.left
    t3 = y.right
    y.right = z
    z.parent = y
    z.left = t3

    if t3 is not None:
        t3.parent = z
    y.parent = sub_root
    if y.parent is None:
        self.root = y
    else:
        if y.parent.left == z:
            y.parent.left = y
        else:
            y.parent.right = y

    z.height = 1 + max(
        self.get_height(z.left),
        self.get_height(z.right)
    )

    y.height = 1 + max(
        self.get_height(y.left),
        self.get_height(y.right)
    )

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def _left_rotate(self, z):
    sub_root = z.parent

```



```

        y = z.right
        t2 = y.left
        y.left = z
        z.parent = y
        z.right = t2

        if t2 is not None:
            t2.parent = z
        y.parent = sub_root

        if y.parent is None:
            self.root = y
        else:
            if y.parent.left == z:
                y.parent.left = y
            else:
                y.parent.right = y

        z.height = 1 + max(
            self.get_height(z.left),
            self.get_height(z.right)
        )
        y.height = 1 + max(
            self.get_height(y.left),
            self.get_height(y.right)
        )

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def get_height(self, no_atual):
    if no_atual is None:
        return -1
    return no_atual.height

# Análise Assintótica Tempo: O(1) Espaço: O(1)
def taller_child(self, no_atual):
    left = self.get_height(no_atual.left)
    right = self.get_height(no_atual.right)
    return no_atual.left if left >= right else no_atual.right

# Análise Assintótica Tempo: O(n) Espaço: O(n + h) - Lista + Pilha
recursão
def transforme_data(self, raiz, data_list):

```

```

        if raiz:
            self.transforme_data(raiz.left, data_list)
            data_list.append({
                "remedio": raiz.remedio,
                "efetivo": raiz.efetivo,
                "colateral": raiz.colateral,
                "seguro": raiz.seguro,
                "passa": raiz.passa
            })
            self.transforme_data(raiz.right, data_list)

# Análise Assintótica Tempo: O(n) Espaço: O(n) - Fila BFS
def bfs(self, raiz):

    if not raiz:
        return False

    fila = deque([raiz])
    passaram = []

    while fila:
        atual = fila.popleft()

        if atual.passa == 1:
            passaram.append(atual.remedio)

        if atual.left:
            fila.append(atual.left)

        if atual.right:
            fila.append(atual.right)

    return passaram

# Análise Assintótica Tempo: O(n) Espaço: O(n + h) - Lista + Pilha
recursão
def dfs(self, raiz, todos_remedios):

    if not raiz:
        return

```

```

        if raiz.remedio:
            todos_remedios.append(raiz.remedio)

        return self.dfs(raiz.left, todos_remedios) or
self.dfs(raiz.right, todos_remedios)

```

Código do arquivo: [grafo.py](#)

```

from collections import defaultdict
from collections import deque

class Grafo:
    # inicialização do grafo
    def __init__(self, vertices):
        self.vertices = vertices
        self.grafo = defaultdict(list)

        for nome, gravidade in self.vertices:
            self.grafo[nome].append(gravidade)

    #Análise Assintótica Tempo: O(n) Espaço: O(1)
    def todos_pacientes(self):
        for vizinhos in self.grafo:
            print(vizinhos)

    #Análise Assintótica Tempo: O(n) Espaço: O(n)
    def alocar_fila_de_pioridade(self, pacientes_prioritarios):
        fila = deque([])
        for paciente in pacientes_prioritarios:
            fila.append(paciente)

        return fila

    #Análise Assintótica Tempo: O(n) Espaço: O(n)
    def mostrar_prioritarios(self, fila):

        for _ in range(len(fila)):
            print(f"Paciente: ", fila.popleft())

    #Análise Assintótica Tempo: O(n) Espaço: O(n)
    def alocar_dados(self):
        data = []

```

```

        for key, value in self.grafo.items():
            data.append({"nome": key, "gravidade": value})

    return data

```

Código do arquivo: [main.py](#)

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn import tree
from avltree import AVLarvore
from grafo import Grafo

if __name__ == "__main__":
    # Criando a árvore para alimentar os dados da rotina de
    # aprendizado de máquina dos remédios.
    avl_tree = AVLarvore()
    dados_para_inserir = []
    vertices = []

    def menu():

        # Coletando dados para arvore AVL
        remedios_quantidade = int(input("Quantos remédios se vai
inserir para análise: "))

        for _ in range(remedios_quantidade):

            print("\n-----\n")
            nome_remedio = input("Coloque o nome do remédio: ")

            eficiencia = input("Remédio é eficiente (s/n):
").lower().strip()

            if eficiencia not in ("s", "n"):
                print("Insira os dados corretamente.")
                return

```

```

        seguro = input("Remédio é seguro de administrar (s/n):
").lower().strip()

        if seguro not in ("s", "n"):
            print("Insira os dados corretamente.")
            return

        colateral = input("Colateral do remédio: (sem/medio/alto):
").lower().strip()

        if colateral not in ("sem", "medio", "alto"):
            print("Insira os dados corretamente.")
            return

        if colateral in ("sem", "medio"):
            colateral_aceitavel = True
        else:
            colateral_aceitavel = False

        if colateral_aceitavel and eficiencia == "s" and seguro ==
"s":
            passa = 1
        else:
            passa = 0

        dados_para_inserir.append((f'{nome_remedio}',
f'{eficiencia}', f'{colateral}', f'{seguro}', passa))

print("\n-----\n")

# Coletando dados para o grafo que vai alimentar a rotina de
aprendizado de máquina dos pacientes.
print("\n===== Coleta de dados dos pacientes
=====\\n")

pacientes_quantidade = int(input("Quantos pacientes
participaram do teste: "))

for _ in range(pacientes_quantidade):

    nome = input("Nome do paciente: ")
    gravidade_input = input("Doença é grave (s/n):
").lower().strip()

```

```

        if gravidade_input not in ("s", "n"):
            print("Insira os dados corretos.")
            return

        gravidade = 1 if gravidade_input == "s" else 0
        vertices.append((nome, gravidade))

    print("\n=====\\n")

    menu()

    # inserindo os dados na avl
    for remedio, efetivo, colateral, seguro, passa in
dados_para_inserir:
        avl_tree.insert(remedio, efetivo, colateral, seguro, passa)

    data_list = []
    # pegar os dados da AVL para fazer o aprendizado de máquina
    avl_tree.transforme_data(avl_tree.root, data_list)

    #dados que vão ser utilizados no aprendizado
    df = pd.DataFrame(data_list)

    inputs = df.drop('passa', axis='columns')
    target = df['passa']

    #transformação de dados que estavam em string para números, pois
máquina apenas ler números
    le_remedio = LabelEncoder()
    le_efetivo = LabelEncoder()
    le_colateral = LabelEncoder()
    le_seguro = LabelEncoder()

    inputs["remedio_n"] = le_remedio.fit_transform(inputs['remedio'])
    inputs["efetivo_n"] =
le_colateral.fit_transform(inputs['efetivo'])
    inputs["colateral_n"] =
le_colateral.fit_transform(inputs['colateral'])
    inputs["seguro_n"] = le_seguro.fit_transform(inputs['seguro'])

    inputs_n = inputs.drop(['remedio', 'efetivo', 'colateral',
'seguro'], axis="columns")

```

```

inputs_sem_numero = inputs.drop(['remedio_n', 'efetivo_n',
'colateral_n', 'seguro_n'], axis="columns")

print("\n ----- Dados -----")
print(inputs_sem_numero)
print("\n ----- Dados Codificados -----")
print(inputs_n)

model = tree.DecisionTreeClassifier()
model.fit(inputs_n, target)

outliers = []
#fazendo a análise dos dados, e mostrando o resultado na tela
print("\n===== Análise de remédios =====\n")
for i in range(len(inputs_n)):
    n1 = inputs_n.iloc[i]["remedio_n"]
    n2 = inputs_n.iloc[i]["efetivo_n"]
    n3 = inputs_n.iloc[i]["colateral_n"]
    n4 = inputs_n.iloc[i]["seguro_n"]

    nome_remedio = inputs.iloc[i]["remedio"]

    # Criando DataFrame para evitar warning
    treinamento_remedios = pd.DataFrame([[n1, n2, n3, n4]],
                                         columns=["remedio_n", "efetivo_n",
"colateral_n", "seguro_n"])
    resultado = model.predict(treinamento_remedios)

    print("=====")
    if resultado == 1:
        print(f"Remédio: {nome_remedio}. Aprovado para venda.")
    else:
        print(f"Remédio: {nome_remedio}. Reprovado, precisa de
mais testes e reajustes a serem feitos.")
        outliers.append((f"Nome do remédio: {nome_remedio}"))
    print("=====")

#pegando os dados que passaram e todos que tiveram no teste.
passaram = avl_tree.bfs(avl_tree.root)
todos_remedios = []
avl_tree.dfs(avl_tree.root, todos_remedios)

```

```

print("\n==== Remédios que passaram no teste =====")
for remedio in passaram:
    print(remedio)

print("\n==== Todos remédios que estão no teste =====")
for remedio in todos_remedios:
    print(remedio)

print("\n==== Outliers detectados durante os testes =====")
for outlier in outliers:
    print(outlier)
print("\n")

# Criando o grafo
grafo = Grafo(vertices)
data = grafo.alocar_dados()

# Dados para a análise
dfpacientes = pd.DataFrame(data)
# utilizamos lambda para transformar o que era [1] ou [0] em 1 ou
0, assim conseguindo fazer o aprendizado de máquina
dfpacientes['gravidade'] = dfpacientes['gravidade'].apply(lambda
x: x[0])

inputs_pacientes = dfpacientes.drop('gravidade', axis="columns")
target_pacientes = dfpacientes['gravidade']

#transformação de string para número
le_nome = LabelEncoder()
inputs_pacientes["nome_numero"] =
le_nome.fit_transform(inputs_pacientes['nome'])

inputs_n_pacientes = inputs_pacientes.drop(['nome'],
axis="columns")

modelPacientes = tree.DecisionTreeClassifier()
modelPacientes.fit(inputs_n_pacientes, target_pacientes)

print("\n==== Tabela dos pacientes ====")
print(inputs_pacientes)

pacientes_prioritarios = []

```



```

#fazendo a análise dos pacientes e o deslocamento para a fila de
prioridades.
print("\n==== Prevendo os pacientes =====")
for i in range(len(inputs_n_pacientes)):
    n = inputs_n_pacientes.iloc[i]["nome_numero"]
    nome = inputs_pacientes.iloc[i]["nome"]

    treinamento_paciente = pd.DataFrame([[n]],
columns=["nome_numero"])
    resultadopacientes =
modelPacientes.predict(treinamento_paciente)

    print(f"\nFazendo o deslocamento de pacientes para fila de
prioritários. Paciente: {nome}\n")
    if resultadopacientes == 1:
        pacientes_prioritarios.append(nome)

fila = grafo.alocar_fila_de_pioridade(pacientes_prioritarios)

print("\n==== Ordem de atendimento dos prioritários =====")
grafo.mostrar_prioritarios(fila)

```

Testes Unitários

Arquivo test_avltree.py:

```

import unittest
from avltree import AVLarvore
class TestAvl(unittest.TestCase):
    def setUp(self):
        print('setUp')
        self.dados = [('A', 's', 'sem colateral', 's', 1)]
        self.avl = AVLarvore()
        for remedio, efetivo, colateral, seguro, passa in self.dados:
            self.avl.insert(remedio, efetivo, colateral, seguro,
passa)

```

```

def test_bfs(self):
    # Com apenas 1 elemento, deve retornar apenas ['A']
    self.assertEqual(self.avl.bfs(self.avl.root), ['A'])

def test_dfs(self):
    todos_remedios = []
    self.avl.dfs(self.avl.root, todos_remedios)
    # Com apenas 1 elemento, deve retornar apenas ['A']
    self.assertEqual(todos_remedios, ['A'])

def test_transforme_data(self):
    data_list = []
    self.avl.transforme_data(self.avl.root, data_list)
    self.assertEqual(data_list, [{'remedio': 'A', 'efetivo': 's',
'colateral': 'sem colateral', 'seguro': 's', 'passa': 1}])

def test_search(self):
    self.assertEqual(self.avl.search(1, 'A'), True)

def test_height(self):
    self.assertEqual(self.avl.height(), 0)

if __name__ == '__main__':
    unittest.main()

```

Arquivo test_grafo.py:

```

import unittest
from grafo import Grafo
from avltree import AVLarvore
import io
import sys
from collections import deque

class TestGrafo(unittest.TestCase):
    def setUp(self):
        print('setUp')
        self.vertices = [("A", 1), ("B", 0), ("C", 1)]
        self.grafo = Grafo(self.vertices)

```

```

def test_todos_pacientes(self):
    captured = io.StringIO()
    sys.stdout = captured
    self.grafo.todos_pacientes()
    sys.stdout = sys.__stdout__

    saida = captured.getvalue().strip().split("\n")
    self.assertEqual(saida, ["A", "B", "C"])

def test_alocar_fila_de_pioridade(self):
    pacientes_prioritarios = ["A", "B", "C"]

self.assertEqual(self.grafo.alocar_fila_de_pioridade(pacientes_priorit
arios), deque(['A', 'B', 'C']))

def test_mostrar_prioritarios(self):
    fila = deque(['A', 'B', 'C'])
    captured = io.StringIO()
    sys.stdout = captured
    self.grafo.mostrar_prioritarios(fila)
    sys.stdout = sys.__stdout__

    saida = captured.getvalue().strip().split("\n")
    self.assertEqual(saida, [
        "Paciente número 1: A",
        "Paciente número 2: B",
        "Paciente número 3: C"
    ])

def test_alocar_dados(self):

    self.assertEqual(self.grafo.alocar_dados(), [{'nome': 'A',
'gravidade': [1]}, {'nome': 'B', 'gravidade': [0]}, {'nome': 'C',
'gravidade': [1]}])

if __name__ == '__main__':
    unittest.main()

```

Referências:

Machine Learning Tutorial Python - 9 Decision Tree. 16 de novembro de 2018. Disponível em: <https://www.youtube.com/watch?v=PHxYNGo8Ncl&t=366s>. Acesso em: setembro de 2025.

AVL Tree: Background & Python Code. 21 de fevereiro de 2018. Disponível em: <https://www.youtube.com/watch?v=lxHF-mVdwK8&t=1s>. Acesso em: setembro de 2025.